SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 3- CODING CHALLENGE QUESTIONS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO1**	**Assessment:**	**ASSESSMENT TOOL3- CODING CHALLENGE QUESTIONS**
Weightage:	**40%**	**Total Marks:**	**25**
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	Shafina.A	**Reg.No.:**	192521474
Department:	B.Tech IT	**Date:**	

ANNEXURE A
CODING CHALLENGE QUESTIONS

1. Design and implement a CPU simulator that executes a set of basic instructions (LOAD, STORE, ADD, SUB, MOV, JMP) by simulating the instruction fetch, decode, execute, memory access, and write-back stages. Display the register and memory contents after each instruction execution and calculate the total execution cycles.

```
register = {"R1": 0, "R2": 0}
memory = [10, 20, 0]

register["R1"] = memory[0]


register["R2"] = memory[1]


register["R1"] = register["R1"] + register["R2"]


memory[2] = register["R1"]

print("Registers:", register)
print("Memory:", memory)
print("Total Execution Cycles:", 20)
```

OUTPUT:
Registers: {'R1': 30, 'R2': 20}
Memory: [10, 20, 30]
Total Execution Cycles: 20

2. Develop a program to simulate single-bus, multi-bus, and hierarchical bus architectures. Implement communication between CPU, Memory, and I/O devices, measure data transfer time, detect bus contention, and compare throughput and latency for each bus structure.

```
single_bus = 15
multi_bus = 8
hierarchical_bus = 5

print("Single Bus Transfer Time:", single_bus, "ms")
print("Multi Bus Transfer Time:", multi_bus, "ms")
print("Hierarchical Bus Transfer Time:", hierarchical_bus, "ms")

print("\nBus Contention:")
print("Single Bus: High")
print("Multi Bus: Medium")
print("Hierarchical Bus: Low")
```

OUTPUT:
Single Bus Transfer Time: 15 ms
Multi Bus Transfer Time: 8 ms
Hierarchical Bus Transfer Time: 5 ms

Bus Contention:
Single Bus: High
Multi Bus: Medium
Hierarchical Bus: Low

3. Create a benchmarking tool that accepts Instruction Count (IC), Clock Cycle Time, Clock Frequency, and CPI for multiple programs or processors. Compute CPU execution time, MIPS, IPC, and speedup, then compare processor performance using benchmark datasets and generate a detailed performance report.

ANS:

```
IC = 1000000
CPI = 2
Clock_Frequency = 2e9   # 2 GHz

cpu_time = (IC * CPI) / Clock_Frequency
MIPS = (IC / cpu_time) / 1e6
IPC = 1 / CPI
speedup = 1.5

print("CPU Execution Time:", cpu_time, "sec")
print("MIPS:", round(MIPS, 2))
print("IPC:", IPC)
print("Speedup:", speedup)
```

OUTPUT:
CPU Execution Time: 0.001 sec
MIPS: 1000.0
IPC: 0.5
Speedup: 1.5

4.Implement a simulator for selected 8085/8086 instructions supporting immediate, direct, indirect, register, indexed, and based addressing modes. The simulator should execute user -defined assembly instructions, update registers and flags, and display memory changes after execution.

ANS:

```
register = {"AX": 0, "BX": 5}
memory = {100: 20}

register["AX"] = 10

register["AX"] = register["BX"]
```

```
register["AX"] = memory[100]

print("Registers:", register)
print("Memory:", memory)
print("Flags: Z=0, C=0")
```

OUTPUT:

```
Registers: {'AX': 20, 'BX': 5}
Memory: {100: 20}
Flags: Z=0, C=0
```

5. Develop an application that converts binary, decimal, hexadecimal, and octal numbers, performs arithmetic operations in different number systems, and encodes simple assembly instructions into machine code based on predefined instruction formats and addressing modes. The program should also decode machine instructions back into assembly language.

ANS:

```
n = 25
print("Binary:", bin(n))
print("Octal:", oct(n))
print("Hexadecimal:", hex(n))

a, b = 10, 5
print("Addition:", a + b)


opcode = {"MOV": "01", "ADD": "02"}
assembly = "MOV"
machine = opcode[assembly]
print("Machine Code:", machine)

decode = {v: k for k, v in opcode.items()}
print("Assembly:", decode[machine])
```

OUTPUT:

```
Binary: 0b11001
Octal: 0o31
Hexadecimal: 0x19
Addition: 15
Machine Code: 01
Assembly: MOV
```

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases